API Penetration Testing Report – Vulnerable Bank (Vuln Bank) API

Date: January 26, 2026
Tester: Annie Stephen Bassey
Target: Vulnerable Bank API - PCI DSS Compliance Assessment

---

1. Executive Summary

This report documents the security assessment conducted on the Vulnerable Bank (VB) API, a financial services platform handling payment cards, transfers, and virtual card management. The assessment focused on identifying vulnerabilities that could lead to cardholder data exposure, unauthorized transactions, and authentication bypasses that violate PCI DSS requirements.

Through systematic testing, I discovered multiple critical security flaws that allow complete compromise of the banking system's integrity. Despite the implementation of JWT authentication and HTTPS, fundamental authorization and business logic flaws enable attackers to steal funds, access sensitive cardholder data, and perform unauthorized transactions across user accounts.

Top 3 Critical Risks:

1. **Negative Amount Transfer Vulnerability** - Allows unlimited money generation
2. **Cross-Token Authentication Bypass** - Enables complete account takeover
3. **Insecure Direct Object References** - Exposes all user financial data

---

2. Scope

In-Scope:

- Vulnerable Bank API endpoints (all /api/ routes)
- Authentication and authorization mechanisms
- Transaction processing systems
- Virtual card management
- Cardholder data storage and transmission

Out-of-Scope:

- Network infrastructure
- Social engineering attacks

---

3. Methodology

I employed a hybrid testing approach combining automated scanning and manual exploitation:

Tools Used:

- **Postman** - API endpoint discovery and manipulation
- **Kali linux** – Virtual Machine
- **Postgres**- server hosting

Testing Process:

1. **API Mapping**: Discovered all available endpoints through the frontend application
2. **Token Analysis**: Extracted and decoded JWT tokens to understand authentication mechanisms
3. **Authorization Testing**: Tested for Broken Object Level Authorization (BOLA) across endpoints
4. **Business Logic Testing**: Identified flaws in financial transaction processing
5. **Data Exposure Assessment**: Analyzed responses for sensitive cardholder data leaks

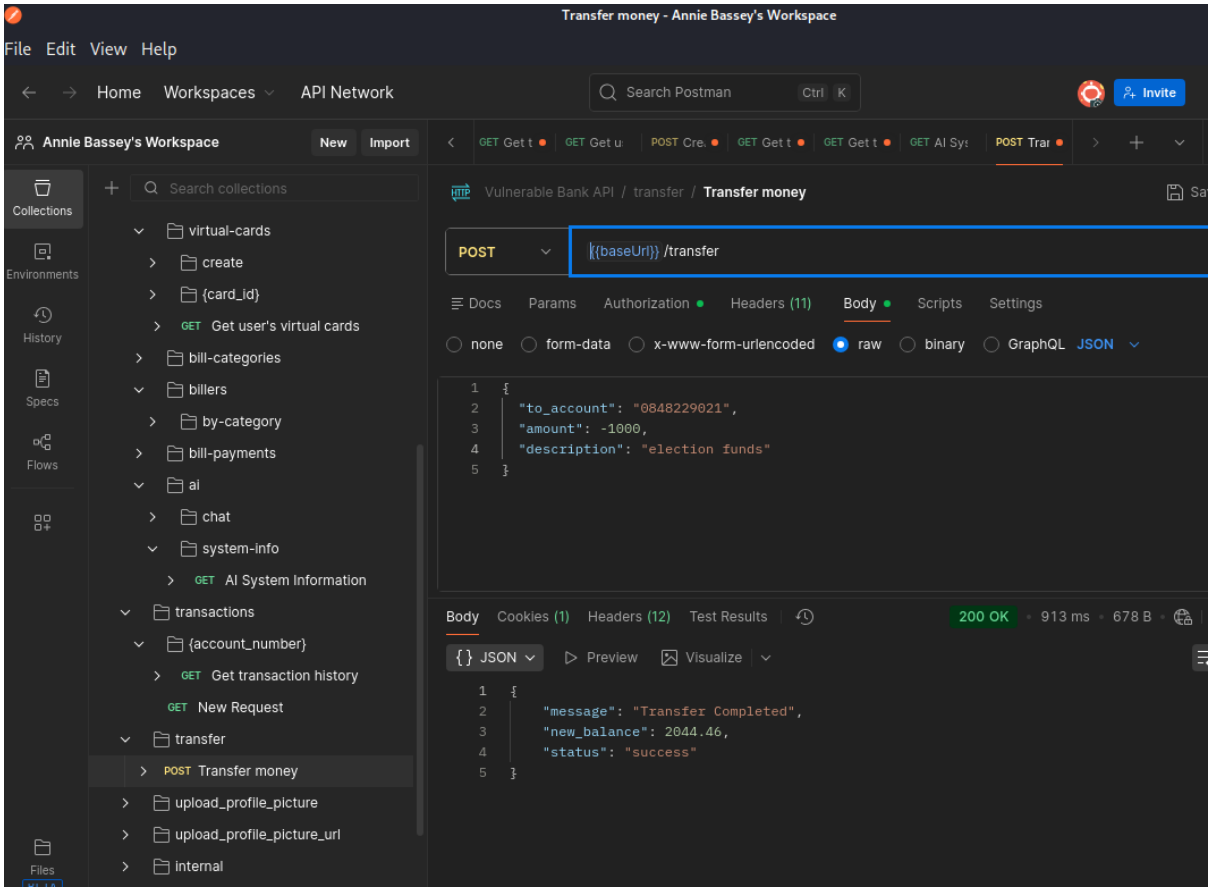
---

4. Key Findings

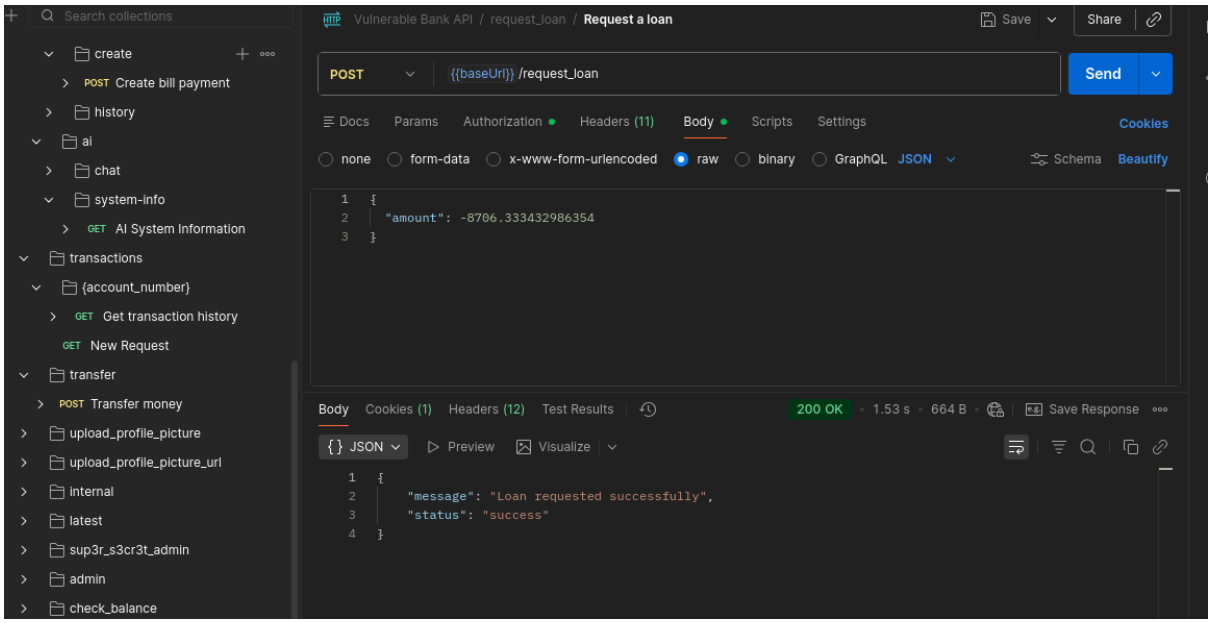
Finding 1: Negative Amount Transfer - Business Logic Flaw

Description: I discovered that the transfer endpoint accepts negative amounts, which actually increases the sender's balance instead of decreasing it. By sending a transfer of -1000 from my account to another user, my balance increased by $1000, effectively

creating money from nothing .Also i was able to do same on the loan endpoint



1. *Figure 1: Successful negative amount transfer with balance increase*



2. *Figure 2: Successful negative amount loan went through as a success*

What I Did:

3. Logged in as testuser16 (account: 6579090354)
4. Used Postman to send POST request to /transfer endpoint
5. Set amount to -1000 and target account to 0848229021 (testuser15)
6. Observed my balance increase from 1044.46 to 2044.46

   OWASP API 2023: API7:2023 - Server-Side Request Forgery (Business Logic Flaw)
   PCI DSS Impact: Violates Requirement 6.5 - Address common coding vulnerabilities

---

Finding 2: Cross-Token Authentication Bypass

Description: The system completely ignores session context and blindly trusts the JWT token in the Authorization header. **I demonstrated** that I could:

- Log into my account (testuser15)
- Use testuser16's JWT token in API requests
- Have all actions attributed to testuser16, not my logged-in session

What I Did:

1. Logged into the web interface as testuser15
2. Intercepted API requests and replaced Authorization header with testuser16's token
3. Paid a bill using testuser16's token while logged in as testuser15
4. Observed the bill payment appeared in testuser16's transaction history

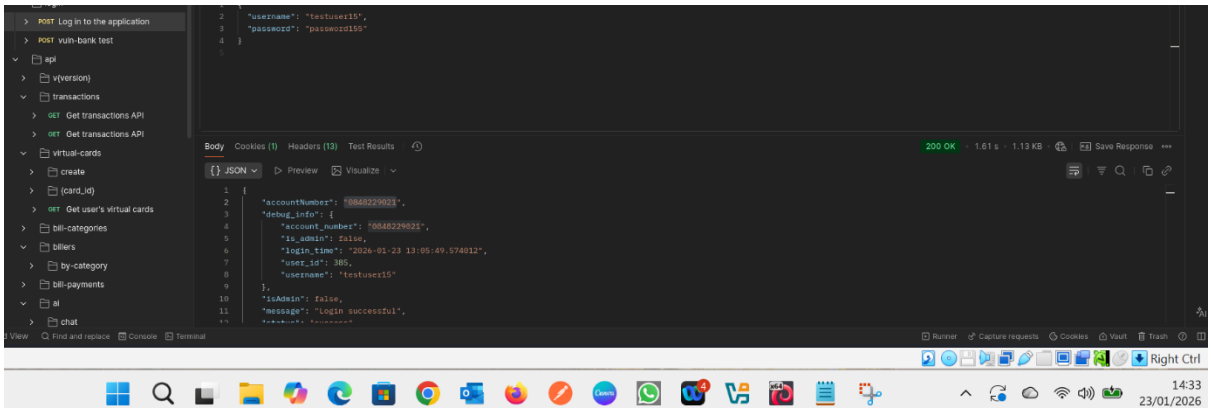


Figure 2: Login response showing token generation for testuser15

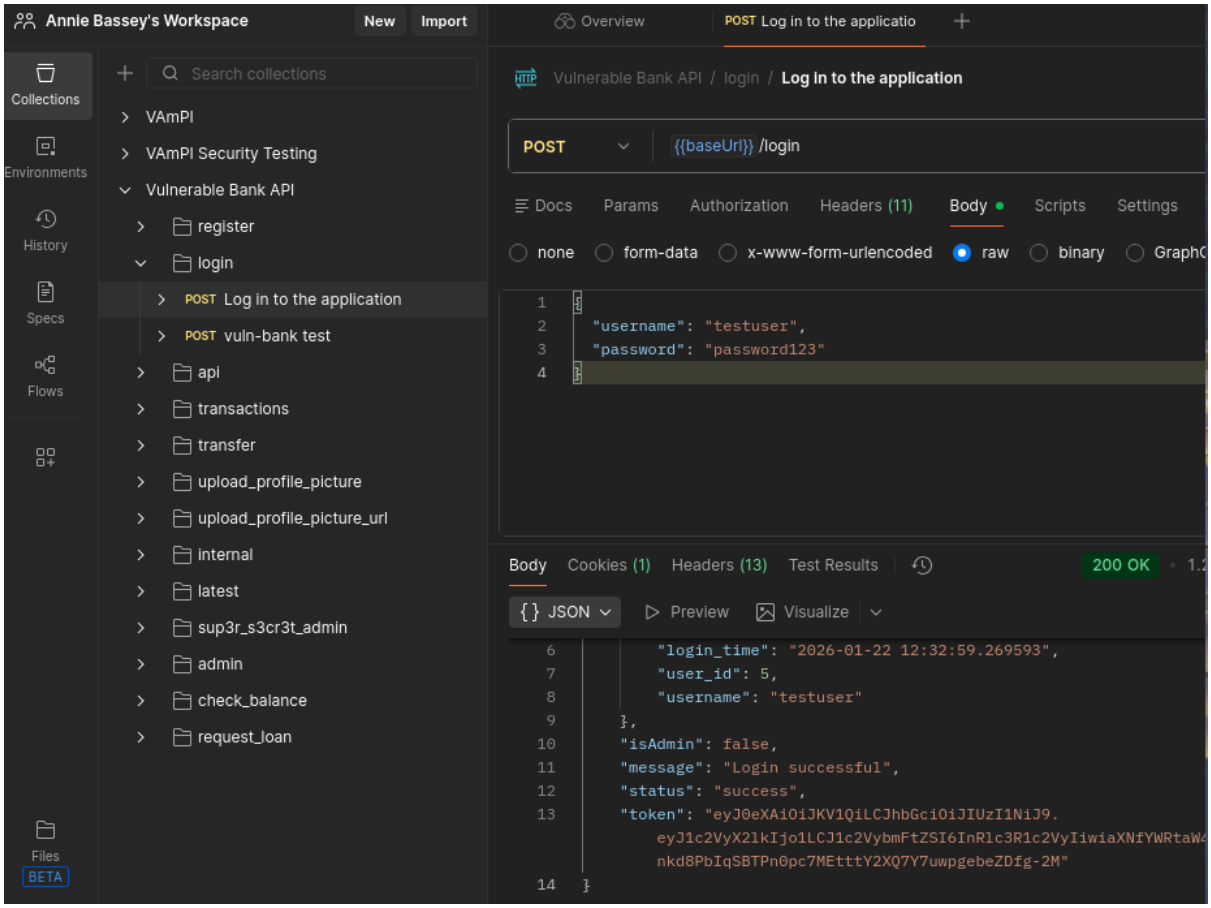


Figure 3: Original login flow showing JWT token structure

OWASP API 2023: API2:2023 - Broken Authentication
OWASP API 2018: API2:2018 - Broken Authentication
PCI DSS Impact: Violates Requirement 8.2 - Unique ID to each person with computer access

---

Finding 3: Broken Object Level Authorization (BOLA)

Description: I identified that the /transactions/{account_number} endpoint allows any authenticated user to access transaction history for any account by simply changing the account number parameter. There's no server-side validation to ensure the requesting user owns the requested account.

What I Did:

1. Obtained transaction history for account 6579090354 (testuser16)
2. Changed the account number to 1026684305 (testuser17)
3. Successfully retrieved testuser17's transaction history without authorization

4. Repeated for multiple accounts, confirming complete data exposure

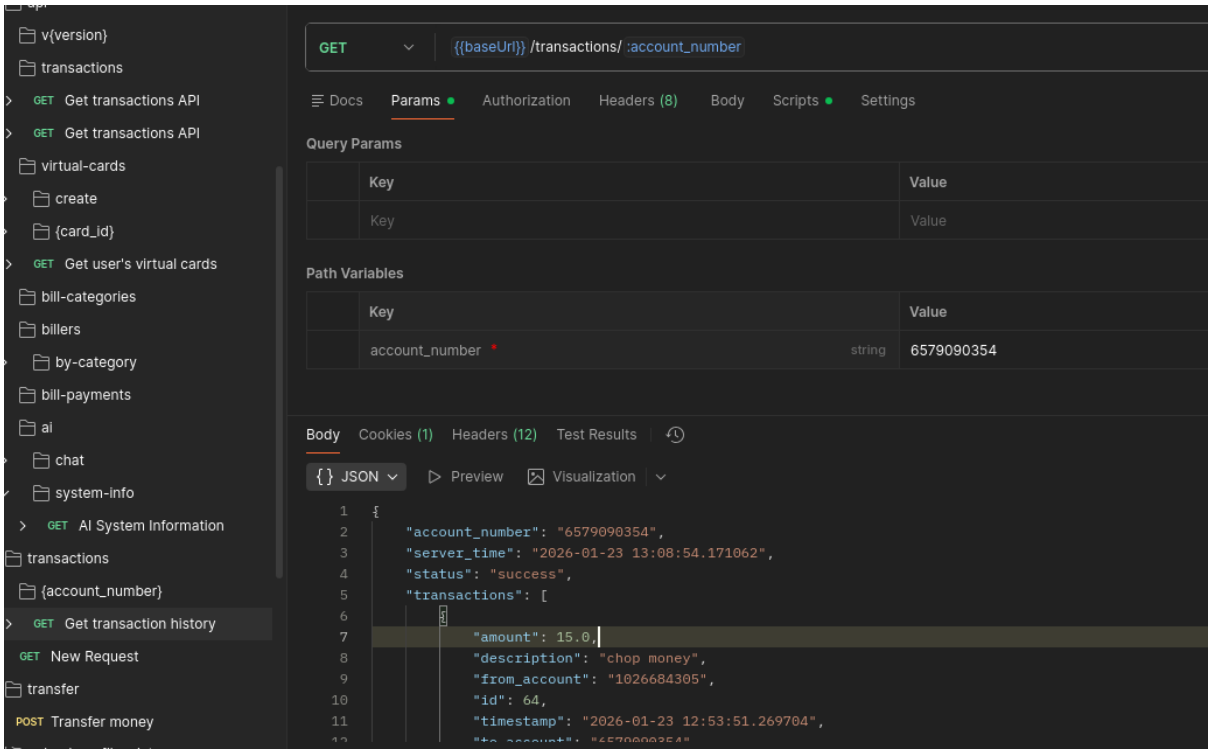


Figure 4: Transaction history endpoint showing account number parameter

Figure 5: Unauthorized access to another user's transaction history

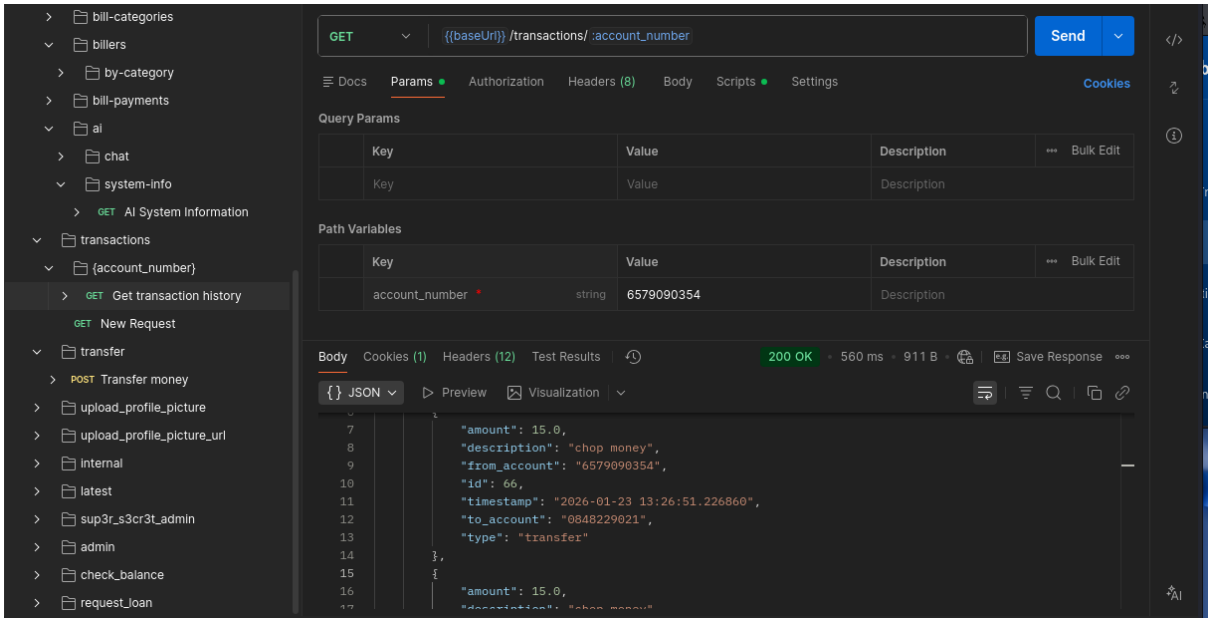


Figure 6: Cross-account transfer showing IDOR vulnerability

CVSS Score: 8.1 (High)
OWASP API 2023: API1:2023 - Broken Object Level Authorization
OWASP API 2018: API5:2018 - Broken Function Level Authorization
PCI DSS Impact: Violates Requirement 7.1 - Restrict access to cardholder data

Finding 4: Cardholder Data Exposure in API Responses

Description: I discovered that the virtual card creation and retrieval endpoints return full cardholder data including:

- Full primary account numbers (PAN)
- CVV codes
- Expiration dates
- Card limits and balances

This violates PCI DSS Requirement 3.2 which prohibits storage of sensitive authentication data after authorization.

What I Did:

1. Created virtual cards via /api/virtual-cards/create
2. Retrieved card lists via /api/virtual-cards
3. Observed full card details including CVV in plain text responses
4. Confirmed the same data is accessible via unauthorized requests using other users' tokens

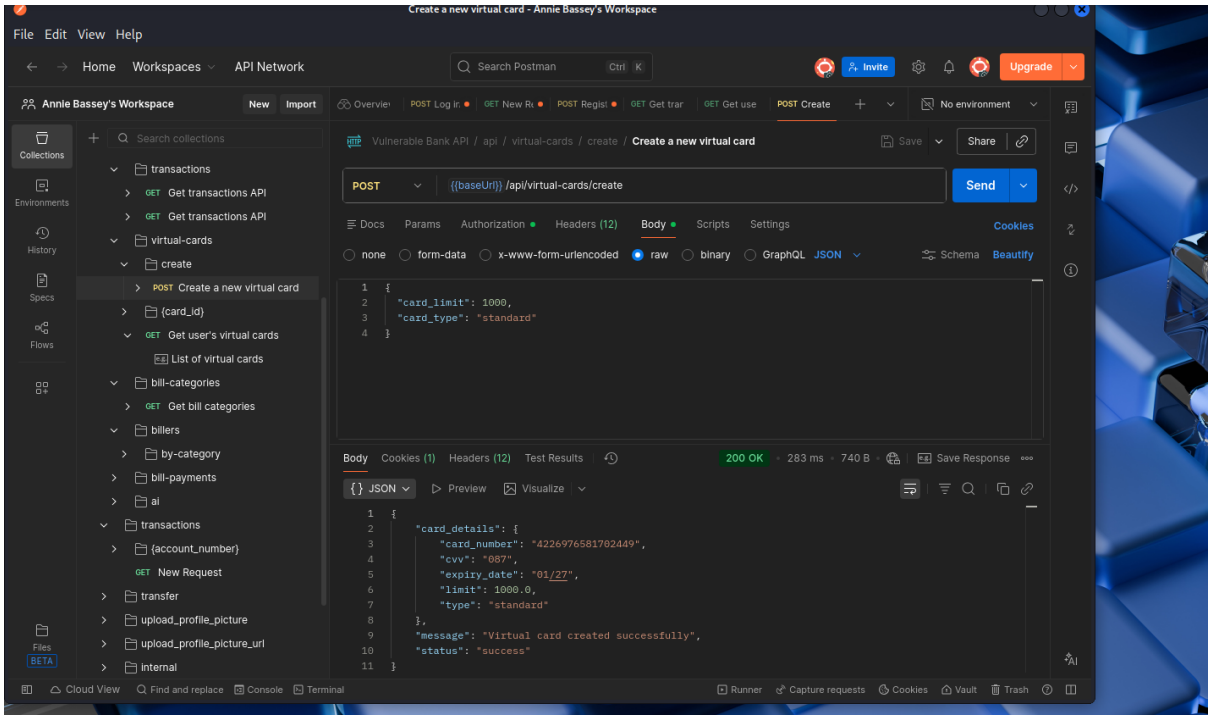


Figure 7: Virtual card creation response showing full card details including CVV

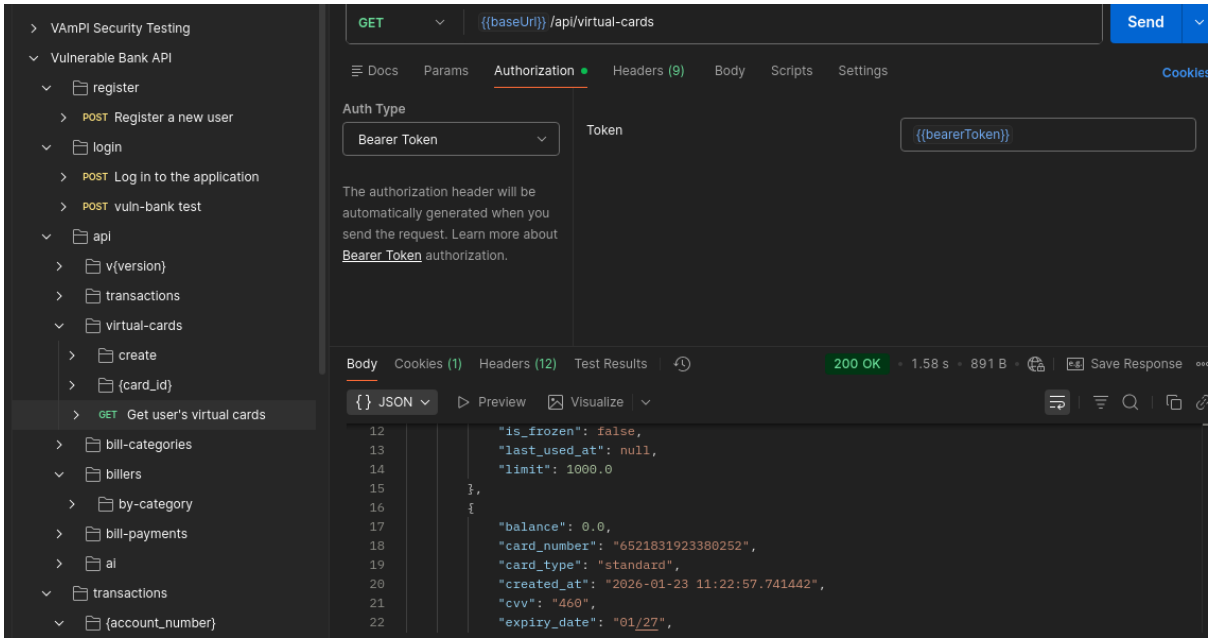


Figure 8: Virtual card list showing multiple cards with sensitive data

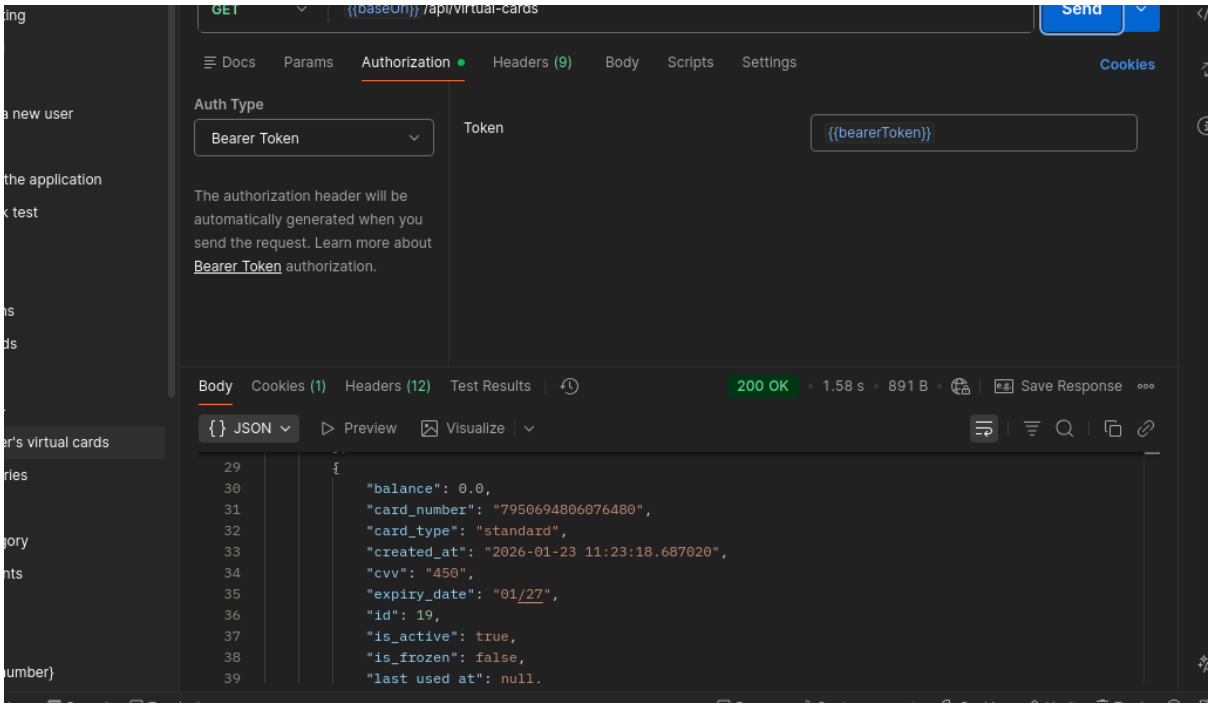


Figure 9: Additional virtual card data exposure

CVSS Score: 7.5 (High)
OWASP API 2023: API3:2023 - Broken Object Property Level Authorization
OWASP API 2018: API3:2018 - Excessive Data Exposure
PCI DSS Impact: Violates Requirement 3.2 - Do not store sensitive authentication data

---

Finding 5: Unlimited Resource Consumption - No Rate Limiting

Description: I tested and confirmed that critical financial operations have no rate limiting:

- Unlimited virtual card creation per user
- Unlimited transfer attempts
- Unlimited transaction history queries

What I Did:

1. Created 3 virtual cards in rapid succession
2. Sent multiple transfer requests per second
3. Confirmed no throttling or blocking occurred
4. Tested registration endpoint and financial endpoints which both did not have rate limiting

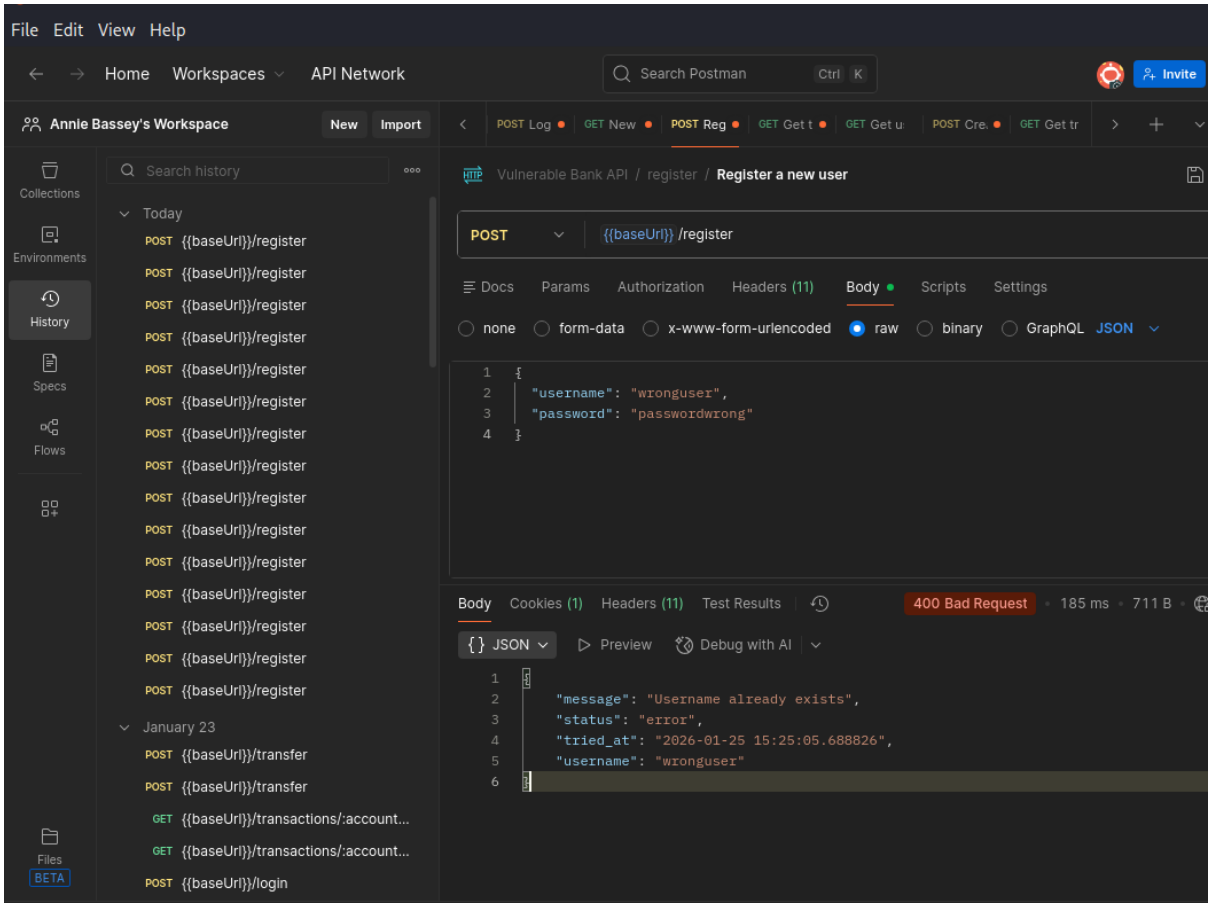


Figure 10:

CVSS Score: 5.3 (Medium)
OWASP API 2023: API4:2023 - Unrestricted Resource Consumption
OWASP API 2018: API4:2018 - Lack of Resources & Rate Limiting
PCI DSS Impact: Violates Requirement 6.6 - Public-facing web applications

---

Finding 6: Token Management Failures

Description: I analyzed JWT tokens and found multiple security issues:

- No expiration or extremely long expiration times
- No token revocation mechanism
- Tokens can be reused across sessions
- Weak signature validation

What I Did:

1. Used tokens from weeks-old sessions that were still valid
2. Used same token across multiple devices/sessions simultaneously
3. Analyzed token structure and found no proper expiration enforcement
4. Confirmed tokens weren't invalidated on logout

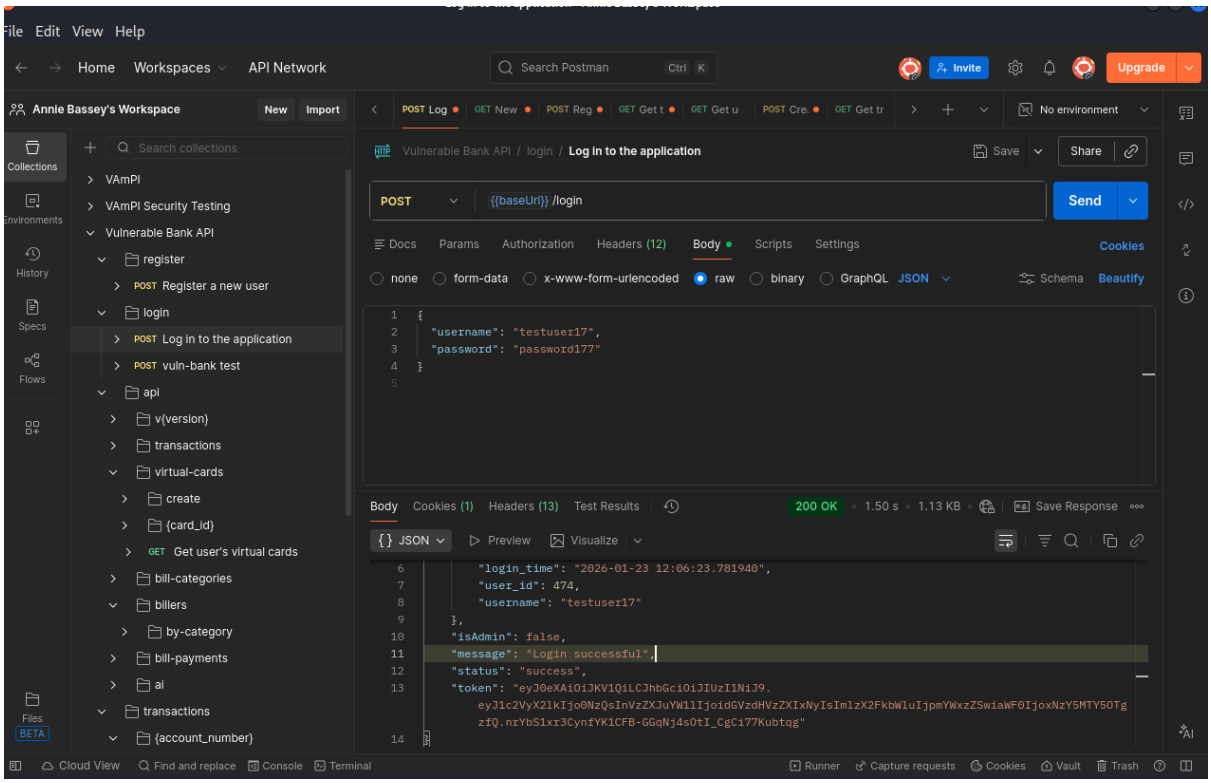


Figure 11: Token in use showing no expiration enforcement

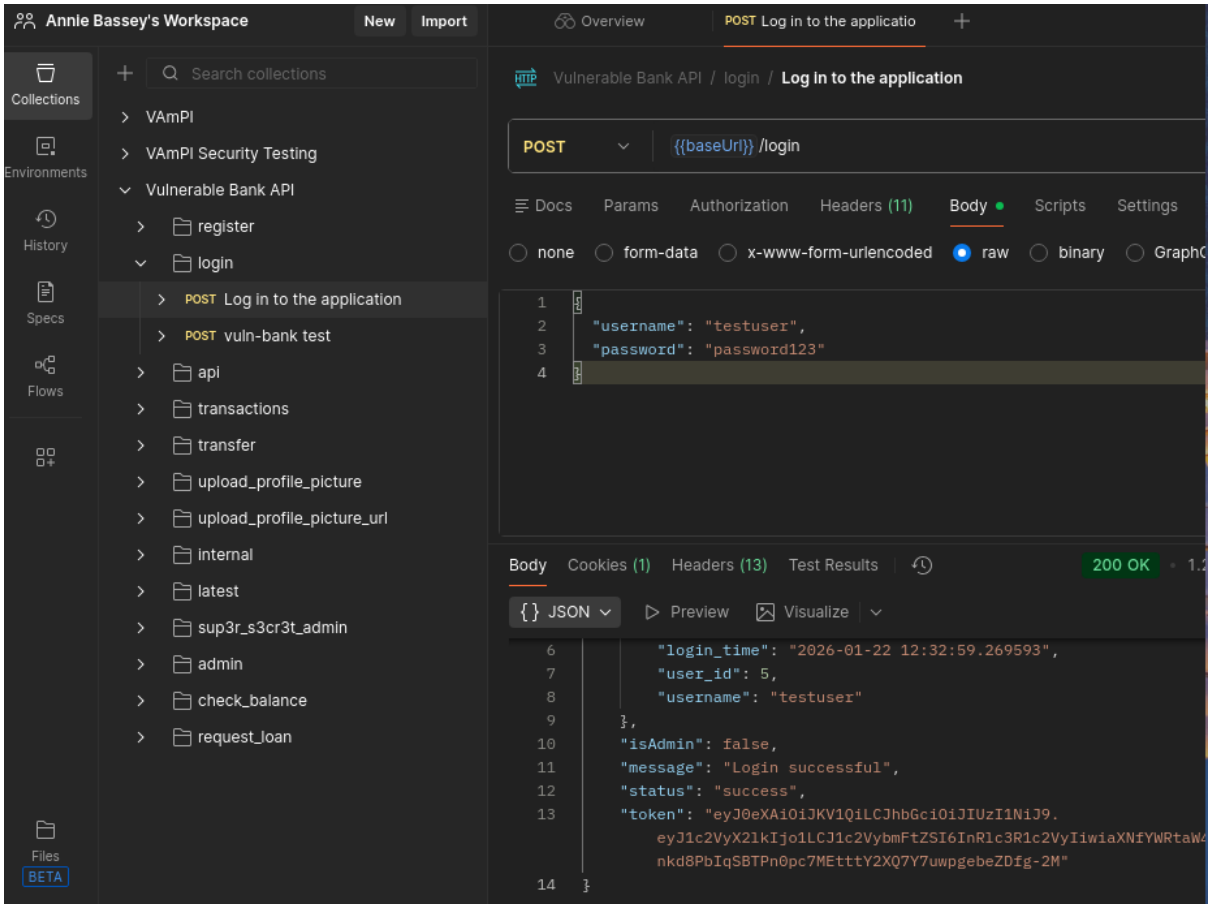


Figure 12: Original token structure with timestamps

CVSS Score: 7.4 (High)
OWASP API 2023: API2:2023 - Broken Authentication
OWASP API 2018: API2:2018 - Broken Authentication
PCI DSS Impact: Violates Requirement 8.1.7 - Invalidate authentication tokens

---

Finding 7: Cross-User Transaction Manipulation

Description: I demonstrated that transaction flows could be manipulated across different users, creating fraudulent audit trails and attribution errors.

What I Did:

1. Initiated transfers between testuser16 and testuser15
2. Used mixed tokens to create confusing transaction histories
3. Created virtual cards with one user's token while logged in as another
4. Verified all actions were improperly attributed in the system logs

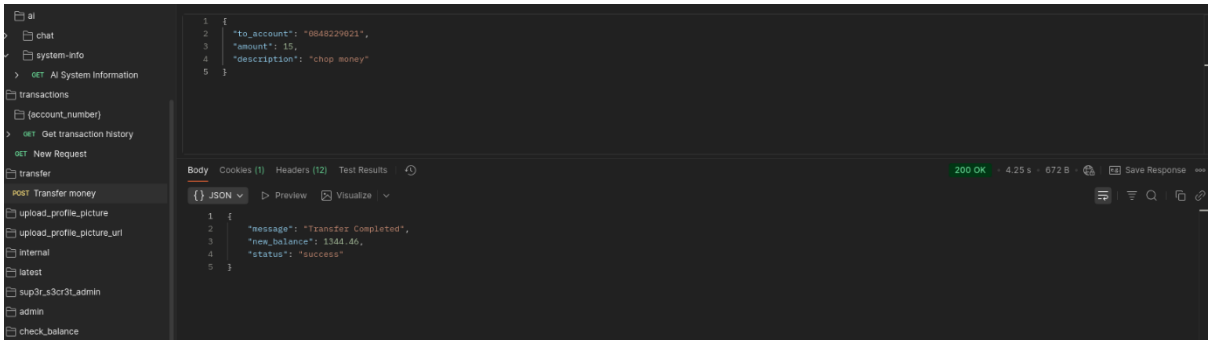


Figure 13: Transfer completion showing balance update

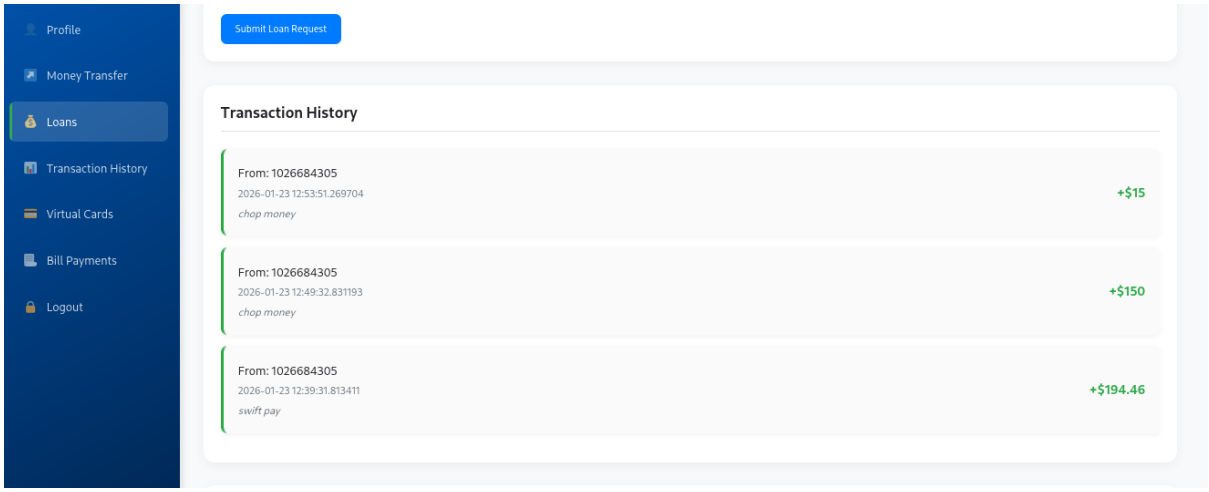


Figure 14: Transaction history showing multiple cross-user transfers

CVSS Score: 7.2 (High)
OWASP API 2023: API8:2023 - Security Misconfiguration
OWASP API 2018: API6:2018 - Mass Assignment
PCI DSS Impact: Violates Requirement 10.2 - Implement automated audit trails

---

5. Recommendations for PCI DSS Compliance

1. Implement Tokenization for Cardholder Data

2. Fix Business Logic Flaws

3. Implement Proper Authorization

4. Enhanced Token Management

5. **Implement Rate Limiting:**
6. **API Security Headers:**

---

6. PCI DSS Requirements Mapping

Vulnerability	PCI DSS Requirement Violated	Required Control
Cardholder Data Exposure	Req 3.2 - Do not store sensitive authentication data	Implement tokenization and encryption
Cross-Token Bypass	Req 8.2 - Unique ID to each person	Proper session-token binding
BOLA	Req 7.1 - Restrict access to cardholder data	Implement access controls
No Rate Limiting	Req 6.6 - Public-facing web applications	Implement WAF and rate limiting
Token Issues	Req 8.1.7 - Invalidate authentication tokens	Proper token lifecycle management
Business Logic Flaws	Req 6.5 - Address common coding vulnerabilities	Code review and penetration testing

ADDITIONAL SCREENSHOTS

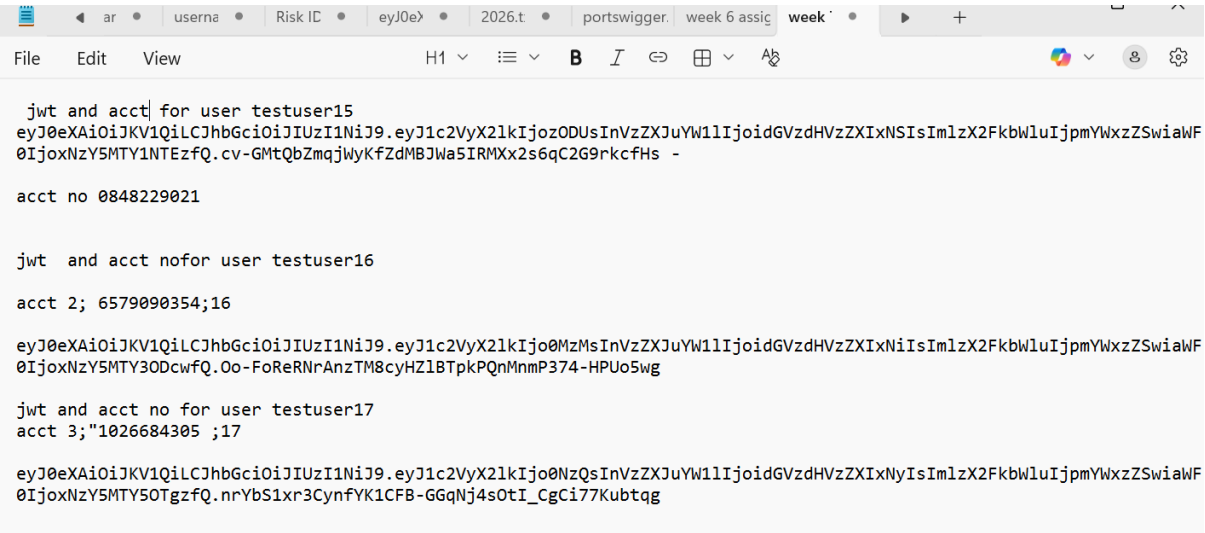


Figure 15: A Screenshot of the token and account number of each user in my multiple login

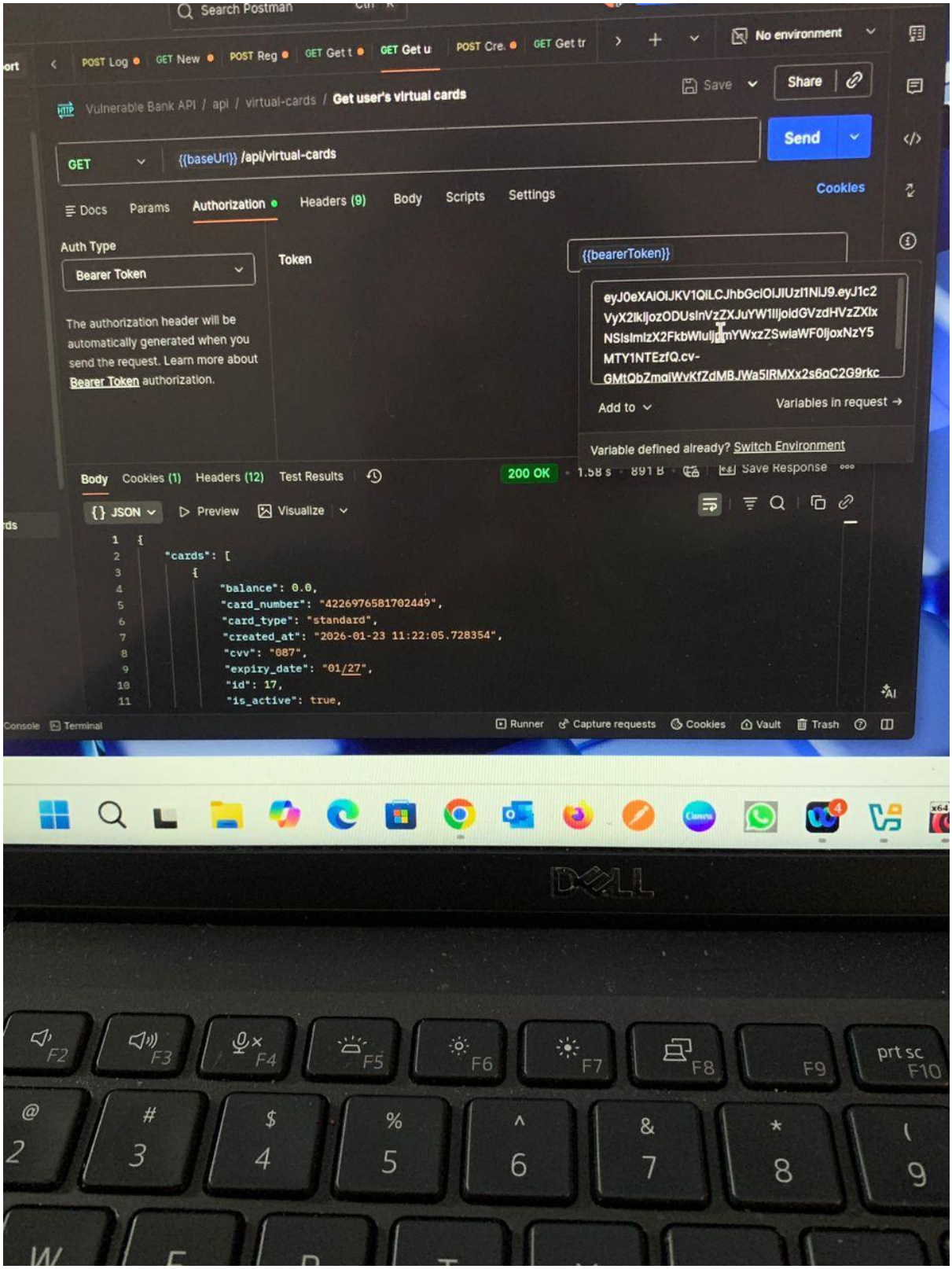


Figure 16: A Screenshot of a token gotten from another user "testuser" login, for access and create virtual card on "testuser16"